# SIMATS ENGINEERING
## CO5-ASSESSMENT TOOL1-Design Task

**COURSE NAME: Artificial Intelligence** **COURSE CODE: CSA1709**

| | |
|---|---|
| **Register Number** | **192311097** |
| **Name** | **Dillilokesh D** |

**COURSE OUTCOME COVERED**

**CO5:** *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications.* (BTL 5)

*Assessment Tool Weightage: 40%*

**Time Duration: 1 Hour**

**QUESTIONS: 5** **Total Marks:50**

***Code of Conduct:***

*I Dillilokesh D 192311097 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***

### 1. End-to-End System Design

Design a scalable AI-based system using **PySpark and RDD operations** to analyze real-time social media data and generate intelligent recommendations, including system architecture, data flow, and agent integration.

### 2. Distributed Intelligent Agent System

Develop a **multi-agent AI system architecture** that uses PySpark for distributed data processing and enables agents to collaborate in solving a large-scale optimization problem.

### 3. Big Data AI Pipeline Design

Design a complete **distributed AI pipeline** using RDD transformations for data preprocessing, feature engineering, and model training, ensuring scalability and fault tolerance.

### 4. Real-Time Decision-Making System

Construct an AI-based system integrating **RDD-based streaming data processing** with intelligent agents for real-time decision-making in applications such as smart traffic or healthcare.

### 5. Cloud-Based Distributed AI Application

Design a **cloud-enabled AI system** using PySpark and intelligent agent architectures that supports large-scale data analytics, dynamic resource allocation, and adaptive learning.

## RUBRICS FOR CALCULATION

| Evaluation Criteria | Problem Understanding & Requirement Analysis | System Architecture Design | Use of PySpark & RDD Operations | Intelligent Agent Integration | Scalability & Distributed Computing Design | Data Flow & Processing Pipeline | Innovation & Creativity | Clarity, Presentation & Documentation |
|---|---|---|---|---|---|---|---|---|
| **Weightage** | 10% | 20% | 15% | 15% | 10% | 10% | 10% | 10% |

| Criteria | Excellent (5) | Good (4) | Average (3) | Poor (2) |
|---|---|---|---|---|
| Problem Understanding & Requirement Analysis | Clearly defines problem, objectives, and constraints | Mostly clear with minor gaps | Basic understanding | Unclear or incorrect |
| System Architecture Design | Complete, scalable, well-structured architecture with diagrams | Good design with minor issues | Basic design, lacks detail | Poor/incomplete |
| Use of PySpark & RDD Operations | Efficient and correct use of RDD transformations/ actions | Mostly correct usage | Limited or partial use | Incorrect/missing |
| Intelligent Agent Integration | Strong integration with clear agent roles and logic | Good integration | Basic integration | Poor/no integration |
| Scalability & Distributed Computing Design | Clearly addresses scalability, fault tolerance, parallelism | Covers most aspects | Limited consideration | Not addressed |
| Data Flow & Processing Pipeline | Clear, logical, and well-defined data flow | Mostly clear | Some gaps | Poorly defined |

| | | | | |
|---|---|---|---|---|
| Innovation & Creativity | Highly innovative, realistic, and impactful solution | Some innovation | Basic idea | No originality |
| Clarity, Presentation & Documentation | Very clear, neat diagrams, well-organized report | Mostly clear | Somewhat organized | Poor presentation |

**ANSWERS:**

# .1. End-to-End System Design: Real-Time Social Media Analytics & Recommendation Engine

## 1.1 Objective

The system ingests high-velocity social media data (posts, comments, likes, shares, hashtags) and produces personalized, low-latency content recommendations. It combines PySpark's distributed RDD/DataFrame engine for processing with a layer of lightweight intelligent agents that reason over the processed signals to rank and personalize recommendations.

## 1.2 High-Level Architecture

```
| Orchestrator)         |
+-----------+-------------+
            v
+-----------------------+
| Serving API + Dashboard |
+-----------------------+
```

## 1.3 Data Flow

1. Posts, comments, and engagement events are published to Kafka topics partitioned by user-region or content category for parallel consumption.
2. Spark Structured Streaming (micro-batch) reads from Kafka; each micro-batch is converted into an RDD of raw JSON records for fine-grained transformation control.
3. RDD transformations clean, tokenize, and enrich records: map for parsing, flatMap for hashtag/token extraction, filter to drop spam/bot traffic, reduceByKey to aggregate engagement counts per content ID, and join to merge user profile RDDs with activity RDDs.
4. A sentiment-scoring agent (a lightweight NLP model broadcast to executors) scores each post; scores are aggregated per topic using reduceByKey for real-time trend detection.
5. Aggregated features are written to a low-latency feature store (Redis for hot keys, Delta Lake for historical replay).
6. A recommendation agent layer combines collaborative filtering (Spark MLlib ALS trained in batch) with a real-time contextual bandit that re-ranks candidates using the freshest streaming features.
7. Final ranked recommendations are exposed through a serving API and pushed to a dashboard for trend monitoring.

## 1.4 Core RDD Operations

```
# Parse and clean incoming events
raw_rdd = kafka_stream_rdd.map(lambda x: json.loads(x[1]))
clean_rdd = raw_rdd.filter(lambda r: r.get("user_id") and not r.get("is_bot"))

# Extract hashtags and compute per-tag engagement
tag_rdd = clean_rdd.flatMap(
    lambda r: [(tag, r["engagement"]) for tag in r.get("hashtags", [])]
)
trend_rdd = tag_rdd.reduceByKey(lambda a, b: a + b)

# Join user activity with user profile for personalization features
joined_rdd = activity_rdd.join(profile_rdd)  # (user_id, (activity, profile))

# Sentiment scoring via broadcast model
bc_model = sc.broadcast(sentiment_model)
scored_rdd = clean_rdd.mapPartitions(
```

```
    lambda part: ((r["post_id"], bc_model.value.score(r["text"])) for r in part)
)
```

## 1.5 Recommendation Agent Integration

- Batch Agent: retrains an ALS matrix-factorization model nightly on the full interaction RDD, persisted to the model registry.
- Streaming Agent: a contextual multi-armed bandit that adjusts candidate ranking using near-real-time engagement and sentiment signals from the feature store.
- Orchestrator Agent: merges candidate lists from both agents, applies diversity/business rules, and returns the final top-N list.

## 1.6 Scalability & Fault Tolerance

- Kafka topic partitioning aligns with Spark RDD partitioning to maximize parallelism and avoid shuffle-heavy joins.
- RDD lineage (DAG) allows automatic recomputation of lost partitions without full pipeline restarts.
- Checkpointing is enabled for stateful streaming aggregations (e.g., rolling trend windows) to bound recovery time.
- Dynamic executor allocation (spark.dynamicAllocation.enabled) scales the cluster with ingestion volume spikes (e.g., viral events).
- Idempotent writes to the feature store (upsert semantics) protect against duplicate processing after task retries.

# 2. Distributed Intelligent Agent System for Large-Scale Optimization

## 2.1 Objective

This architecture uses PySpark as the distributed compute substrate for a multi-agent system that collaboratively solves a large-scale combinatorial optimization problem — for example, vehicle routing, warehouse resource allocation, or distributed job scheduling — where the search space is too large for a single machine to explore sequentially.

## 2.2 Agent Roles

| Agent | Responsibility | Spark Mechanism |
|---|---|---|
| Partitioner Agent | Splits the global search space / problem instance into independent sub-problems | sc.parallelize + custom Partitioner |
| Explorer Agents | Search assigned sub-space (e.g., local search, genetic mutation, simulated annealing) | RDD.mapPartitions |
| Evaluator Agent | Scores candidate solutions against the global objective/constraints | RDD.map + broadcast objective fn |
| Coordinator Agent | Aggregates best local solutions, decides next iteration or convergence | RDD.reduce / treeAggregate |
| Memory (Blackboard) Agent | Maintains shared best-known solution and constraint state | Broadcast variable + Accumulator |

## 2.3 Architecture Diagram

```
|  re-broadcast next round)  |
+---------------------------+
```

## 2.4 Iterative Optimization Loop

1. The Partitioner Agent divides the problem instance (e.g., N delivery routes, M warehouse tasks) into P partitions, one per Spark task, using sc.parallelize(problem_chunks, numSlices=P).
2. Each Explorer Agent, running inside mapPartitions, performs local optimization (hill-climbing, simulated annealing, or genetic operators) on its chunk, using a broadcast copy of the current global best solution as a seed/reference.
3. Local candidate solutions are scored by the Evaluator Agent logic embedded in the same partition pass (avoiding an extra shuffle), returning (candidate, score) pairs.
4. The Coordinator Agent aggregates results with a treeAggregate (more scalable than a flat reduce for large partition counts) to find the global best candidate for this iteration.
5. The Blackboard state (a broadcast variable) is updated and re-broadcast for the next iteration; an Accumulator tracks convergence metrics (e.g., improvement delta, iterations without improvement).
6. The loop terminates when the Coordinator detects convergence (score plateau) or a fixed iteration budget is exhausted.

## 2.5 Code Sketch

```
def explorer_partition(partition, bc_global_state):
    local_best = None
    for chunk in partition:
        candidate = local_search(chunk, bc_global_state.value)
        score = evaluate(candidate)
        if local_best is None or score > local_best[1]:
            local_best = (candidate, score)
    yield local_best

state_bc = sc.broadcast(initial_state)
improvement_acc = sc.accumulator(0.0)

for iteration in range(max_iterations):
    results_rdd = problem_rdd.mapPartitions(
        lambda part: explorer_partition(part, state_bc)
    )
    global_best = results_rdd.treeAggregate(
        (None, float("-inf")),
        seqOp=lambda acc, x: x if x[1] > acc[1] else acc,
        combOp=lambda a, b: a if a[1] > b[1] else b,
    )
    delta = global_best[1] - state_bc.value.score
```

```
improvement_acc.add(max(delta, 0))
state_bc = sc.broadcast(update_state(global_best))
if delta < CONVERGENCE_THRESHOLD:
    break
```

## 2.6 Collaboration & Communication Patterns

- Broadcast variables act as the shared 'blackboard' so every agent reads the same global state without repeated shuffles.
- Accumulators provide a lightweight, write-only channel for agents to report progress metrics back to the Coordinator.
- treeAggregate reduces communication overhead versus a single-stage reduce when the number of partitions/agents is large, avoiding driver bottlenecks.
- For problems requiring agent-to-agent negotiation (not just aggregation), an auxiliary message bus (Kafka topic per agent group) can be layered in for asynchronous bidding/negotiation protocols.

## 2.7 Fault Tolerance & Scalability

- RDD lineage re-executes only the lost partitions (i.e., only the affected Explorer Agents), not the whole optimization run.
- Checkpointing the Blackboard state every K iterations bounds recovery cost for long-running optimizations.
- Speculative execution mitigates straggler agents exploring unusually large or hard sub-spaces.

# 3. Big Data AI Pipeline Design: RDD-Based Preprocessing, Feature Engineering & Training

## 3.1 Objective

A complete, fault-tolerant pipeline that transforms raw, messy big data into a trained, evaluated machine learning model, built primarily on RDD transformations for full control over partitioning and low-level processing, with Spark MLlib for the training stage.

## 3.2 Pipeline Stages


## 3.3 Stage-by-Stage RDD Transformations

Stage 1 — Ingestion

```
raw_rdd = sc.textFile("s3a://data-lake/raw/events/*.json", minPartitions=200)
parsed_rdd = raw_rdd.map(json.loads)
```

Stage 2 — Cleaning & Validation

```
valid_rdd = (
    parsed_rdd
    .filter(lambda r: r.get("timestamp") is not None)
    .filter(lambda r: all(k in r for k in REQUIRED_FIELDS))
    .map(lambda r: normalize_types(r))
    .distinct()  # de-duplicate
)
```

Stage 3 — Feature Engineering

```
# Numeric scaling stats computed via aggregate (single pass, fault-tolerant)
sum_count = valid_rdd.map(lambda r: (r["value"], 1)) \
                     .reduce(lambda a, b: (a[0]+b[0], a[1]+b[1]))
mean_val = sum_count[0] / sum_count[1]

feature_rdd = valid_rdd.map(lambda r: build_feature_vector(r, mean_val))

# Categorical encoding via broadcast lookup table
bc_vocab = sc.broadcast(build_vocab(valid_rdd))
encoded_rdd = feature_rdd.map(lambda f: one_hot_encode(f, bc_vocab.value))
```

Stage 4 — Model Training (MLlib on RDD API)

```
from pyspark.mllib.regression import LabeledPoint
from pyspark.mllib.tree import RandomForest

labeled_rdd = encoded_rdd.map(lambda f: LabeledPoint(f.label, f.features))
train_rdd, test_rdd = labeled_rdd.randomSplit([0.8, 0.2], seed=42)
train_rdd.persist()  # cache for iterative training

model = RandomForest.trainClassifier(
    train_rdd, numClasses=2, categoricalFeaturesInfo={},
    numTrees=100, featureSubsetStrategy="auto", maxDepth=8
)
```

Stage 5 — Evaluation

```
predictions_rdd = test_rdd.map(lambda lp: (model.predict(lp.features), lp.label))
correct = predictions_rdd.filter(lambda pl: pl[0] == pl[1]).count()
accuracy = correct / test_rdd.count()
```

Stage 6 — Deployment

- Serialized model is written to the model registry (e.g., S3 + metadata DB) with a version tag and evaluation metrics.
- A serving layer (Spark batch job or a lightweight REST wrapper loading the model) performs inference on new data.

## 3.4 Scalability Techniques

| Technique | Purpose |
|---|---|
| Partition tuning (repartition/coalesce) | Balance parallelism against shuffle cost per stage |
| persist()/cache() | Avoid recomputation of the training RDD across iterative algorithm passes |
| Broadcast joins for small lookup tables | Avoid expensive shuffle joins for vocab/reference data |
| treeAggregate / treeReduce | Reduce driver bottleneck when aggregating statistics across many partitions |
| Columnar storage (Parquet) for source data | Faster I/O and predicate pushdown at ingestion |

## 3.5 Fault Tolerance

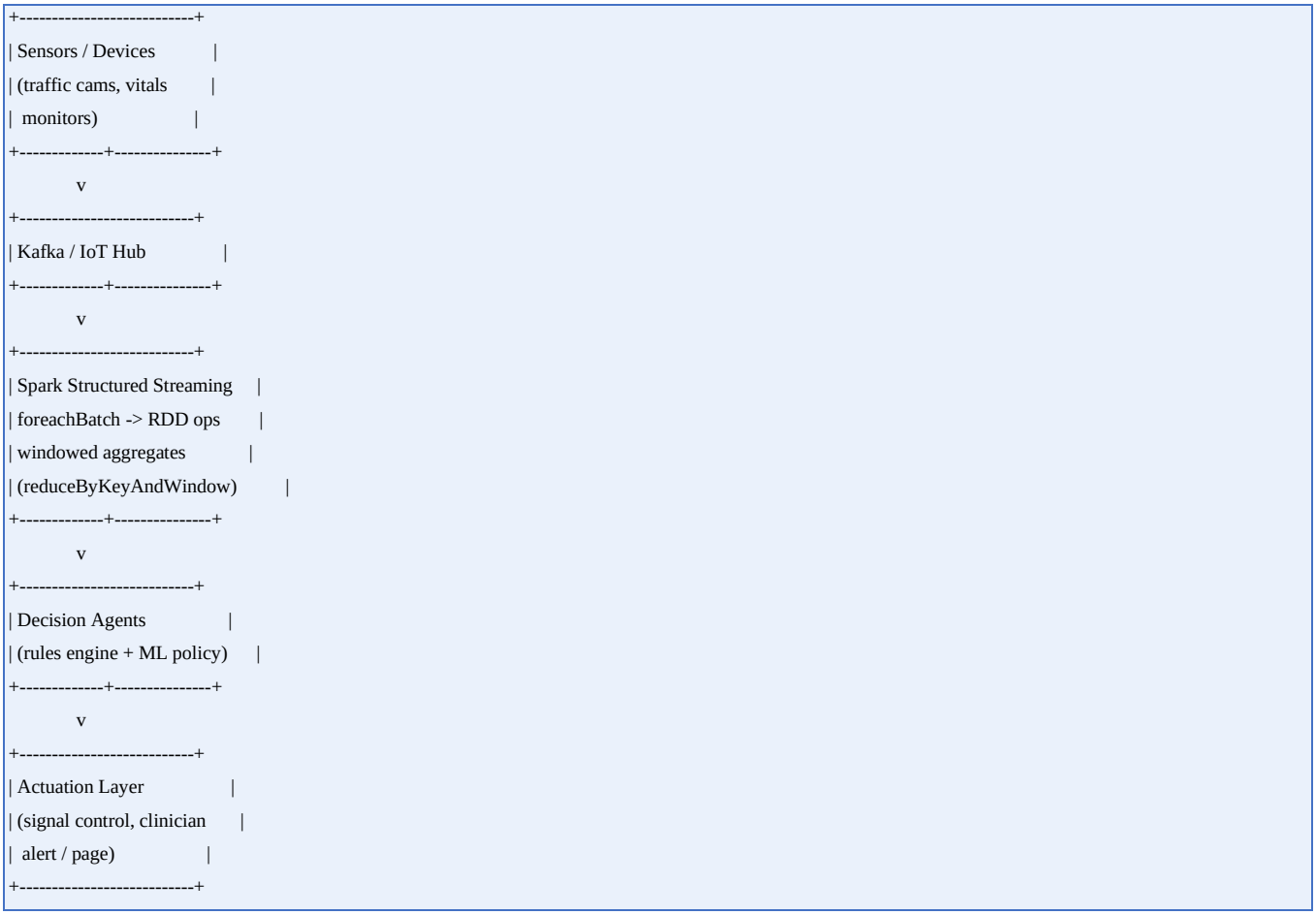
- RDD lineage graphs allow Spark to recompute only lost partitions after executor failure, without restarting the whole pipeline.
- Checkpointing the cleaned/feature RDDs truncates long lineage chains for iterative training algorithms, reducing recovery time.
- Idempotent, atomic writes (write-to-temp then rename/commit) protect the model registry from partial-write corruption on task retries.
- Speculative execution re-launches slow tasks on healthy executors to mitigate stragglers without manual intervention.

# 4. Real-Time Decision-Making System: Streaming RDDs + Intelligent Agents

## 4.1 Objective

A low-latency system that ingests continuous sensor/event streams, processes them with RDD-based micro-batch operations, and feeds intelligent agents that make and act on real-time decisions. Two applications are addressed: smart traffic-signal control and healthcare patient-monitoring alerts.

## 4.2 General Architecture




## 4.3 Use Case A — Smart Traffic Signal Control

1. Traffic cameras/sensors at each intersection stream vehicle-count and speed events to Kafka, partitioned by intersection ID.
2. Spark Structured Streaming consumes micro-batches; foreachBatch exposes each batch as an RDD for custom windowed aggregation of congestion levels per intersection.
3. A per-intersection Signal-Control Agent (a small rules engine or reinforcement-learning policy) consumes the aggregated congestion metrics and decides green-light duration adjustments.

4. Decisions are pushed to an actuation topic consumed by edge controllers at each intersection; a Coordination Agent monitors corridor-level (multi-intersection) flow to avoid conflicting local decisions (e.g., green-wave synchronization).

```
def process_batch(batch_df, batch_id):
    rdd = batch_df.rdd
    congestion_rdd = (
        rdd.map(lambda r: (r.intersection_id, r.vehicle_count))
           .reduceByKey(lambda a, b: a + b)
    )
    decisions = congestion_rdd.map(lambda kv: signal_agent.decide(kv[0], kv[1]))
    decisions.foreachPartition(publish_to_actuation_topic)

streaming_df.writeStream.foreachBatch(process_batch).start()
```

## 4.4 Use Case B — Healthcare Real-Time Monitoring

1. Wearable/bedside monitors stream vitals (heart rate, SpO2, blood pressure) to Kafka, partitioned by patient/unit.
2. Streaming RDD transformations compute rolling windows per patient (reduceByKeyAndWindow) to smooth noise and detect trend deviations.
3. A Patient-Risk Agent scores each window against clinical thresholds and a broadcast early-warning model (e.g., a logistic model approximating a National Early Warning Score).
4. High-risk scores trigger an Alert Agent that pages the responsible clinician through an integration layer, with an audit trail written for compliance.

*Health-related automation of this kind should be designed and validated with qualified clinical and regulatory oversight before any real deployment; this design covers the data/AI architecture only, not clinical validation or certification.*

## 4.5 Latency & Correctness Controls

| Concern | Mechanism |
|---|---|
| Late/out-of-order events | Watermarking on event-time with a bounded lateness threshold |
| Exactly-once decisioning | Idempotent writes keyed by (entity_id, window_id) to the actuation/alert topic |
| Bounded decision latency | Small micro-batch interval (e.g., 1-2s) with backpressure enabled |
| State growth | TTL-based state expiry for inactive intersections/patients |
| Agent failure isolation | Per-entity agent state checkpointed independently to limit blast radius |

## 4.6 Fault Tolerance

- Structured Streaming checkpoints (offsets + state) allow exact resumption after driver or executor failure.
- Write-ahead logs on the Kafka source guarantee no event loss between failures and recovery.

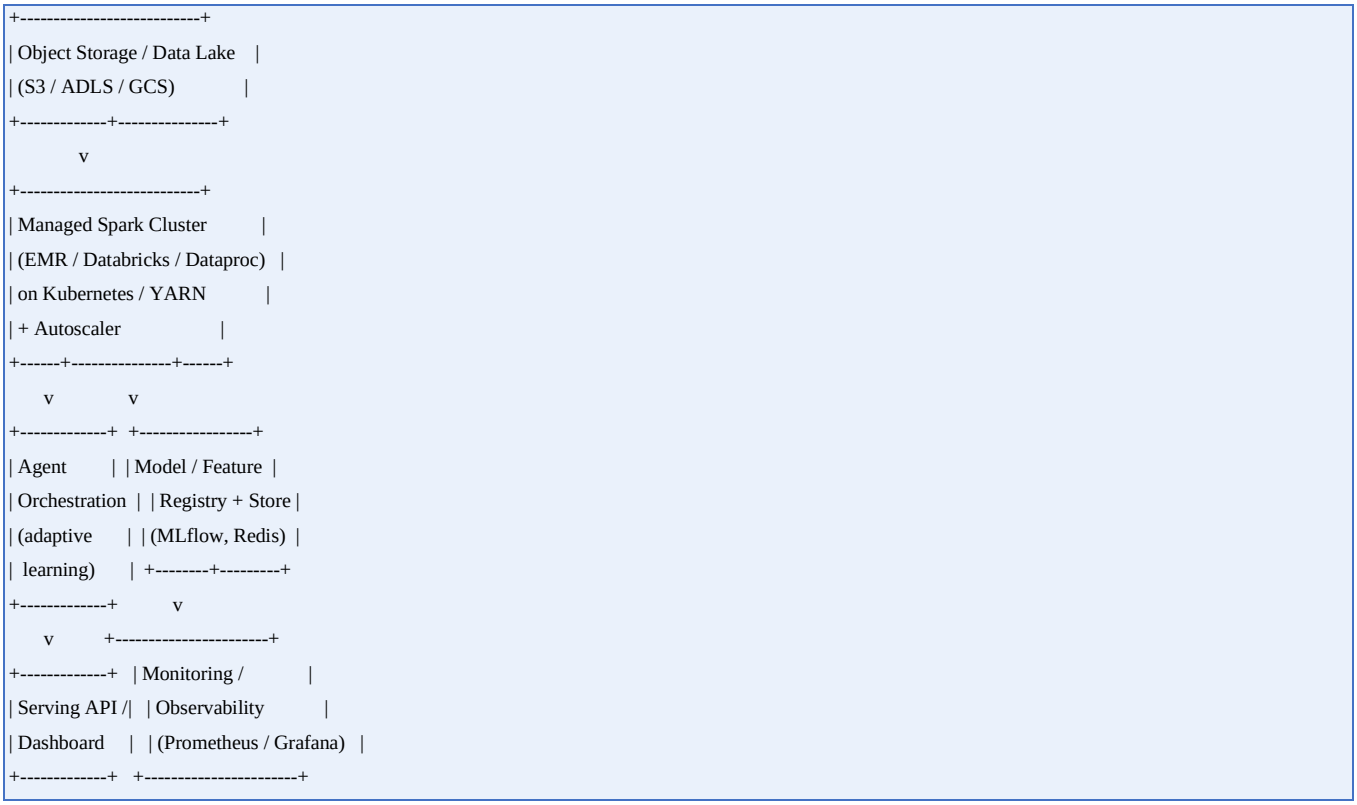
- Agents are stateless per decision cycle where possible, re-deriving context from the feature store rather than holding fragile in-memory state.

## 5. Cloud-Based Distributed AI Application: Elastic PySpark + Adaptive Agents

### 5.1 Objective

A cloud-native system that runs PySpark workloads with intelligent agents for large-scale analytics, elastic (dynamic) resource allocation, and adaptive/online learning — deployable on managed Spark services (Databricks, AWS EMR, GCP Dataproc) or self-managed Spark-on-Kubernetes.

### 5.2 Cloud Architecture




### 5.3 Dynamic Resource Allocation

Spark's dynamic allocation lets the cluster grow and shrink executors based on real-time workload, avoiding both over-provisioning cost and under-provisioning latency:

```
spark = SparkSession.builder \
    .appName("cloud-adaptive-ai") \
    .config("spark.dynamicAllocation.enabled", "true") \
    .config("spark.dynamicAllocation.minExecutors", "4") \
    .config("spark.dynamicAllocation.maxExecutors", "200") \
    .config("spark.dynamicAllocation.executorIdleTimeout", "60s") \
```

```
.config("spark.shuffle.service.enabled", "true") \
.getOrCreate()
```

- Kubernetes/YARN-level autoscaling adds or removes nodes beneath Spark's executor allocation for coarse-grained elasticity.
- Spot/preemptible instances can be used for stateless Explorer/Evaluator-style agent tasks to cut cost, with checkpointing protecting against pre-emption.

## 5.4 Adaptive Learning Agents

1. A Monitoring Agent continuously tracks model performance (drift, accuracy decay) on live data using a sliding-window evaluation RDD/DataFrame.
2. When drift exceeds a threshold, a Retraining Agent triggers an incremental or full retraining job (auto-scaled Spark job) using the latest feature store snapshot.
3. A Champion/Challenger Agent runs the newly trained model alongside the production model on a shadow traffic split, promoting the challenger only if it outperforms on live metrics.
4. A Deployment Agent registers the promoted model in the registry and triggers a rolling update of the serving layer, with automatic rollback on regression.

## 5.5 Observability & Governance

| Layer | Tooling / Approach |
|---|---|
| Cluster metrics | Prometheus + Grafana dashboards for executor utilization, shuffle spill, GC time |
| Data quality | Automated schema/statistics checks on each batch before it enters the pipeline |
| Model governance | MLflow (or equivalent) model registry with lineage, versioning, and approval gates |
| Cost monitoring | Cloud billing tags per job/agent + cluster autoscaling alerts on runaway usage |
| Security | IAM-scoped access to storage/registry; encrypted data at rest and in transit |

## 5.6 Why RDDs Still Matter in a Cloud/DataFrame World

While DataFrame/Structured Streaming APIs are preferred for most production analytics due to the Catalyst optimizer, the underlying RDD API remains the right tool for: custom partitioning strategies (as used by the agent systems above), fine-grained control needed by iterative optimization and reinforcement-learning-style agents, and low-level operations (mapPartitions, broadcast/accumulator patterns) that DataFrame APIs do not expose directly. This design combines DataFrame-based ingestion/ETL with RDD-level control at the agent decision points where it matters most.